



UNIVERSIDAD DON BOSCO
FACULTAD DE INGENIERIA
ESCUELA DE COMPUTACION

CICLO
II

GUIA DE LABORATORIO #06

Nombre de la Practica: POO: Implementación de Herencia
Lugar de Ejecución: Centro de Computo
Tiempo Estimado: 2 horas y 30 minutos
Materia: Desarrollo de aplicaciones con software propietario

I. OBJETIVOS

Que el estudiante:

- Determine cuando utilizar la Herencia de Objetos dentro de los requerimientos de un Programa
- Diseñe las clases principales y clases derivadas, necesarias para implementar la Herencia
- Implemente correctamente el principio del Polimorfismo

II. INTRODUCCION TEORICA

HERENCIA Y POLIMORFISMO

El manejo de propiedades, métodos y eventos es sólo el principio de la programación orientada a objetos; las características principales de la **POO** descansan en la capacidad de los objetos para trabajar en colaboración con otros objetos, y de aprovechar total o parcialmente su funcionalidad mediante la reutilización de código según lo requieran las aplicaciones.

A continuación, se estudiará los principios que permiten el máximo aprovechamiento del código a través de POO y con sus dos capacidades características: **herencia y polimorfismo**.

Herencia

La herencia (inheritance) es un concepto fundamental de la programación orientada a objetos, y **consiste en la capacidad de definir clases que sirven como base de otras clases, evitando la redundancia en la definición de elementos como métodos, eventos y propiedades**.

La herencia permite crear nuevas clases partiendo de clases existentes, que tendrá todas los atributos, propiedades y los métodos de su 'superclase' o 'clase padre' y además se le podrán añadir otros atributos, propiedades y métodos propios.

Hay una clase base y una o varias clases derivadas (o subclases); las clases derivadas aprovechan todo lo declarado en la clase base, asumiéndolo como suyo y utilizando capacidades que no están codificadas en ellas.

Las clases derivadas pueden heredar, e incluso extender, las propiedades, los métodos y los eventos de la clase base. Es posible también que las clases derivadas sustituyan o modifiquen (override) la funcionalidad de los métodos de la clase base.

Las ventajas de la herencia son muchas, pero la principal es que el desarrollador tendrá que escribir y depurar el código sólo una vez, para una clase, y disponer en muchas clases de la misma funcionalidad, estableciendo las referencias adecuadas.

¿Cuándo se debe utilizar la herencia?

La herencia entre clases no debe utilizarse por el simple hecho de que el lenguaje lo permite. La herencia debe planearse; el desarrollador debe pensar cómo han de ser las clases, organizándolas en lo que se conoce como **Jerarquía de clases**.

Se recomienda una jerarquía de clases cuando entre las clases se presenta una relación de tipo **Es-un (is - a)**; debe evitarse el uso de herencia cuando se trata de relaciones **Tiene-un (has - a)**.

Por ejemplo, una clase que contiene las características de los automóviles en general (Automóviles) tiene mucho que compartirle a una clase que almacena las particularidades de los automóviles deportivos (AutomóvilesDeportivos), pues un automóvil deportivo, antes que ser de tipo deportivo, es un automóvil (is-a).

Ahora pregúntese, ¿qué comparte un dueño de un automóvil y un automóvil? Automóviles y DueñosAutomóviles, no comparten rasgos. Las personas que poseen automóviles no se derivan de los automóviles, ni viceversa. Que una persona tenga un (has-a) automóvil, no quiere decir que comparta con éste sus métodos, propiedades y eventos. Imagínese tener que determinar el tipo de transmisión, número de puertas y número de serie que una persona posee. Simplemente, de una cosa no puede derivar la otra.

Se recomienda una jerarquía de clases, obviamente, sólo si el código de una clase brinda alguna funcionalidad que se pueda reutilizar; tratar de heredar aspectos muy particulares que sólo son pertinentes para una clase no tiene sentido.

Clase Base y Clase Derivada

La herencia permite crear una nueva clase que se vuelve a utilizar, se extiende y modifica el comportamiento que se define en otra clase.

La clase cuyos miembros se heredan se denomina la Clase Base.

La clase que hereda esos miembros recibidos de su clase Base, se denomina la Clase Derivada..

Importante:

- Todas las clases de C# heredan implícitamente los miembros de la clase [Object]
- Los lenguajes administrados en el .NET Framework no admiten la herencia múltiple. En otras palabras, puede especificarse sólo una clase base para una clase derivada.

Operador de herencia (:)

El operador (:) a continuación del nombre de una clase, permite indicar el nombre de la clase Base de la cual hereda sus miembros públicos existentes.

La clase nueva es la clase derivada (NombreClaseDerivada); la clase ya existente es la clase base (NombreClaseBase). La sintaxis es la siguiente:

```
public class NombreClaseDerivada : NombreClaseBase
{
    ...
}
```

En este caso, la clase derivada dispone de todos los elementos declarados en la clase base.

Niveles de accesibilidad

Para controlar el grado/nivel de acceso a los miembros de una clase base (padre o contenedora) o derivada, se definen los modificadores de acceso: `public`, `protected`, `internal` o `private`. El funcionamiento de cada uno se indica a continuación:

Accesibilidad declarada	Significado
public	Acceso no restringido
protected	Acceso limitado a la clase contenedora o a los tipos derivados de esta clase. No podrá accesarse desde el exterior de un objeto de una clase derivada.
internal	Acceso limitado al ensamblado actual
protected internal	Acceso limitado al ensamblado actual o a los tipos derivados de la clase contenedora
private	Acceso limitado al tipo contenedor

Sólo se permite un modificador de acceso para un miembro o tipo, excepto cuando se utiliza la combinación **protected internal**.

Clase no heredable (sealed)

Como ya se mencionó, por omisión todas las clases en C# pueden heredarse.

Pero habrá ocasiones en que una clase no deba ser usada como una clase base para otras. En tal caso, al declarar una clase, se utiliza la palabra **sealed**. La sintaxis para ello es la siguiente:

```
public sealed class NombreClase
{
    ...
}
```

En este caso, al intentar heredar NombreClase generaría un error (**InheritsNombreClase**).

De esta manera, una clase se puede declarar como sellada, la cual no podrá ser usada como una clase base ni como una clase abstracta.

Clase abstracta (palabra reservada **abstract)**

Como sabemos, los objetos son instancias de una clase. Como se ha mencionado antes, no se puede tener acceso a la funcionalidad de una clase sino es a través de una instancia de la misma, es decir, de un objeto basado en la clase. Eso es verdadero sólo relativamente.

¿Habrá alguna forma de que podamos disponer de la funcionalidad de una clase sin generar una instancia de esta?

La respuesta es que sí. Podemos declarar una clase que no pueda instanciarse pero que pueda ser utilizada sólo si se hereda por otra clase. Para hacer eso es necesario especificar la palabra clave **abstract** al definir la clase.

La sintaxis es la siguiente:

```
public abstract class NombreClaseHeredable
{
    ...
}
```

En este caso, instanciar un objeto de la clase NombreClaseHeredable generaría error.

Estatutos auxiliares para el manejo de herencia y polimorfismo

Por omisión, una clase derivada hereda los métodos y propiedades de su clase base

Pero en ocasiones, al heredar de una clase, nos damos cuenta que algunas entidades (métodos, propiedades, eventos) no se ajustan del todo a lo que verdaderamente necesitamos; en resumen, no nos sirven tal y como fueron heredadas.

Puede configurarse que una propiedad o método heredado se comporte de manera distinta a como se definió en la clase base o ser remplazada por completo.

Dichas modificaciones se realizan en la clase derivada, respecto de las definiciones originales existentes en la clase base.

La programación orientada a objetos hace posible que eso suceda, pues permite implementar diferentes comportamientos para un método (**virtual**), o ser remplazarlo por completo (**abstract**).

La funcionalidad heredada puede modificarse por otra que sea más indicada para el problema de programación que deseamos resolver; con ello se promueve que la reutilización del código sea flexible y adaptable.

III. MATERIALES Y EQUIPO

Para la realización de la guía de práctica se requerirá lo siguiente:

No.	Requerimiento	Cantidad
1	Guía de Laboratorio de DSP404	1
2	PC con Microsoft Visual Studio 2015 .NET o posterior	1
3	Memoria USB	1
4	Computadora con acceso a Internet	1

IV. PROCEDIMIENTO

Para esta práctica deberá crear una carpeta con el nombre de “Guia6_SuCarnet”, en la cual guardara todos los archivos de esta práctica.

Parte I: Análisis de un Problema de Herencia de Clases

1. A continuación, desarrollara la implementación de la solución al siguiente problema:

PROBLEMA A RESOLVER

Se requiere registrar la información básica de un total de N doctores y de M vendedores. Cada uno de ambos tipos de empleados tiene información en común y otros datos que son diferentes para cada uno.

El sistema debe permitir a usuario asignar una foto para cada uno y mostrar la información ambos listados de empleados cuando usuario lo desee.

2. Analizando la información de cada tipo de empleado, se ha desarrollado un conjunto de 3 clases, en donde una de ellas contiene la información común a ambos tipos de empleados y las otras clases heredan esta información y se especializan con miembros específicos.

El diagrama de clase (en UML) se muestra en la Figura 1.

Observe que la clase principal/base es la clsPersona y las otras 2 clases (clsDoctor y clsVendedor), son clases hijas/derivadas, que adquieren los miembros de clsPersona y c/u implementa sus propios miembros

- Se ofrece una propuesta de solución. A continuación, desarrollara la implementación de la solución al siguiente problema:

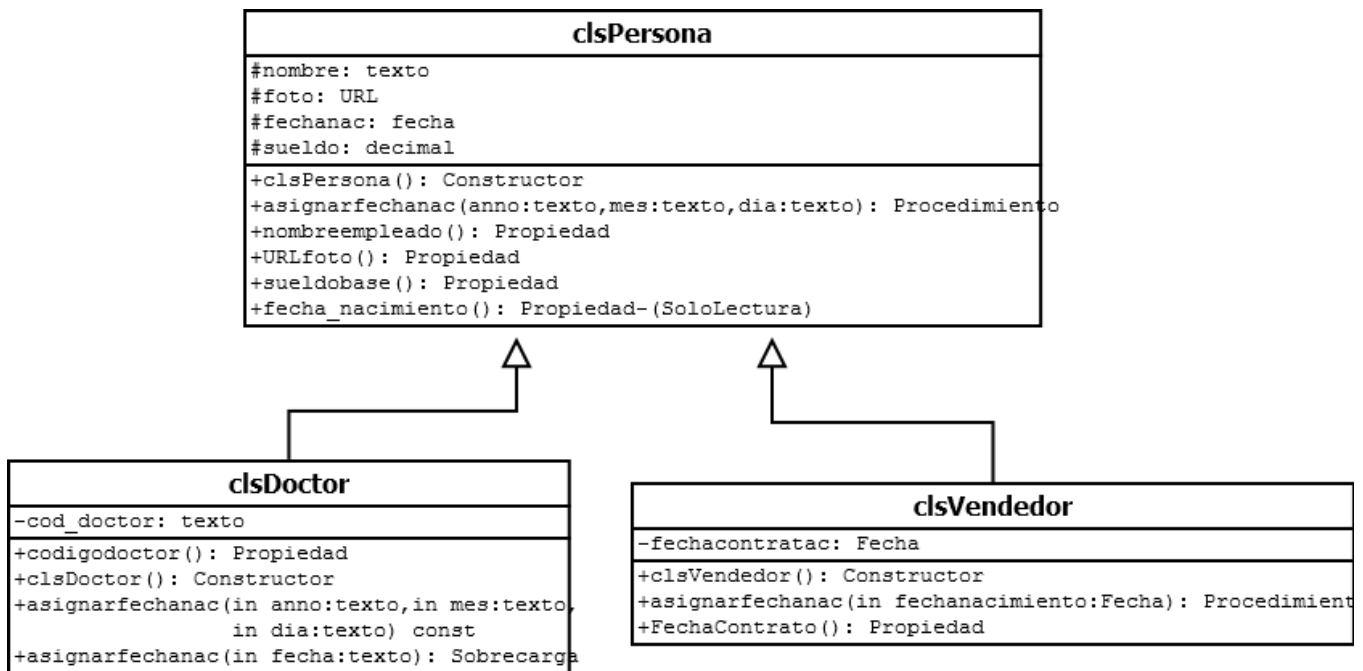


Figura 1: Diagrama UML de las 3 clases a implementar en la aplicación bajo C#

Parte II: Preparando el entorno grafico de la aplicación

- Proceda a descargar el proyecto base (Guia6Ejercicio1) proporcionado junto a la Guía de Practica 6 (Localizar en el segmento **Recursos**).
- Abra el proyecto descargado en el paso anterior y comience a ejecutar los cambios en los objetos y propiedades indicados a continuación.
- Desde la ventana de Propiedades, selección al control TabControl1. De su listado de propiedades, ubicar y dar clic en la colección TabPages. En esta ventana, haga estos cambios:
 - A la ficha tabPage1, cambie su propiedad Text por "Ingreso de Doctores".
 - Seleccione el miembro tabPage2 y modifique su propiedad Text por "Ingreso de Vendedores"
 - Presione botón Agregar, para agregar una nueva ficha (TabPage3). Modifique su propiedad Text por "Ver empleados registrados".
 - Presione botón Aceptar.
- Sobre este control TabControl1, de clic en el titulo de la 3er ficha. En esta área de trabajo vacía, coloque 2 nuevos controles (RadioButton).
Luego, seleccione el DataGridView1 ya existente en el form y arrástrelo sobre el área de esta nueva ficha, para que este se integre a la 3era ficha (igual como lo hicieron los 2 RadioButton).

8. De la caja de propiedades, seleccione en el combo al control TabControl1. Luego, coloque el ratón sobre la “cruz de flechas” que aparece en la esquina superior izquierda de este control y arrástrelo hasta centrarlo en el form.

De manera similar, en el área de la 3era ficha de este control, distribuya apropiadamente al datagridview1 y los 2 radio button.

9. Modifique las propiedades indicadas por cada control a continuación:

Control	Text
groupBox1	Foto
groupBox2	Fecha de nacimiento
label1	Nombre completo
label2	Seleccione foto...
label3	Sueldo base (\$)
label4	Cod. Medico
label5	Nombre completo
label6	Seleccione archivo de foto...
label7	Fecha Nacimiento
label8	Fecha Contratacion

Control	name
textBox1	txtNom
textBox2	txtNomV
dateTimePicker1	dtpFechaNac
dateTimePicker2	dtpFechaC

Control	name	DropDowsStyle
comboBox1	cmbDia	DropDownList
comboBox2	cmbMes	DropDownList
comboBox3	cmbAnno	DropDownList

Control	Name	Mask
maskedTextBox1	mtbCodDoctor	AA-000000

Control	name	Text
button1	btnBuscar	Buscar
button2	btnAgregarDoc	Agregar a Doctor
button3	btnBuscar2	Buscar
button4	btnAgregarVen	Agregar a Vendedor
radioButton1	rbDoctores	Ver doctores registrados
radioButton2	rbVendedores	Ver vendedores registrados

Control	name	SizeMode
pictureBox1	picFotoDoc	StretchImage
pictureBox2	picFotoVen	StretchImage

Control	name	Maximum y Minimum	DecimalPlaces
NumericUpDown1	nudSueldo	10000 y 0.0	2

10. Guarde los cambios hasta ahora y compile su programa.

Haga un reconocimiento de cada una de las 3 fichas y los controles en cada una.

Parte 2: Diseño de las clases a implementar

- Desde la ventana del Explorador de Soluciones, ubique el nombre del espacio de nombres del proyecto abierto (Guia6Ejercicio1) y de clic secundario sobre él.

Del menú contextual, seleccione Agregar -> Clase...

En la ventana emergente, localice en la parte inferior al nombre del archivo y asignar como nombre del archivo (.cs) a su número de carnet.

- Se abre una nueva ventana de edición de código, con las referencias a un conjunto de namespace heredados y una extensión del namespace del proyecto actual.
- Con nuevas sentencias **using**, agregue a los espacios de nombres **System.Windows.Forms** y también a **System.IO**

Este ultimo namespace permitirá ejecutar tareas de acceso a los archivos/carpetas del SO.

- Cambie el nombre de la clase predetermina ahí por el de **clsPersona**. Y coloque el cursor a continuación de su llave de apertura.

Ahí digite la siguiente definición de miembros de esta nueva clase:

```
//CAMPOS
//(protected: indica que seran visibles solo en interior de
//los miembros de esta clase base y sus clases derivadas)
protected string nombre;

protected string foto; //URL de foto seleccionada para empleado

protected DateTime fechanac;
protected float sueldo;

//METODOS

public clsPersona() //constructor
{
    nombre = "";
    foto = ""; //url de foto-empleado aun no asignada
    fechanac = Convert.ToDateTime("1900/01/01"); //fecha predeterminada
}

public virtual void asignarfechanac(string anno, string mes, string dia){
/*
Este metodo debera ser reemplazado en clases derivadas, con otro metodo
que tenga el mismo encabezado (lista de parametros o firma)
*/
    fechanac=Convert.ToDateTime( anno+"/"+mes+"/"+dia);
    MessageBox.Show("de clase Persona");
}

//PROPIEDADES
```



```

public string nombreempleado
{
    //accede a propiedad: nombre
    get{
        return(nombre);
    }
    set{
        if(value.Length>1)
            nombre=value;
        else
            nombre="";
    }
}

public string URLfoto { //accede a la URL del archivo JPG elegido por la persona
    get
    {
        return (foto); //retorna la URL de donde se selecciono foto
    }
    set
    { //recibe la url de la imagen seleccionada por usuario
        if (nombre == "")
        {
            MessageBox.Show("usuario no tiene aun nombre asignado");
        }
        else
        {
            //genera la URL del nuevo archivo JPG, que se copiara hacia carpeta de esta aplicacion
            foto = Application.StartupPath + "\\\" + nombre + ".jpg";

            //Copia archivo de ubic. elegida por usuario hacia ruta del ejecutable del proyecto actual
            File.Copy(value, foto, true);
        }
    }
}

public float sueldobase
{
    //acceso lectura-escritura hacia campo sueldo
    get
    {
        return (sueldo);
    }
    set {
        if (value> 0)
            sueldo = value;
        else
            MessageBox.Show("Valor de sueldo incorrecto", "ERROR-SISTEMA", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}

public string fecha_nacimiento
{ //propiedad solo-lectura
    get
    {
        return (fechanac.ToShortDateString().ToString());
    }
}

```

15. Ubique el cursor justamente despues del cierre de la definición de la clase anterior, para luego redactar la siguiente definición de la clase clsDoctor, la cual heredara miembros de la clase clsPersona.

```
class clsDoctor : clsPersona
{
//campo privado: solo accesibles en interior de esta clase
string cod_doctor;

//Propiedades

public string codigodoctor
{
get {
return (cod_doctor);
}
set {
cod_doctor = value;
}
}

//Metodos

public clsDoctor() { //metodo constructor
nombre = "";
sueldo = 5;
fechanac = Convert.ToDateTime("1900/1/1");
}

//Este metodo en la clase Hija es el reemplazo exigido por la clase Base
//->Observe que tiene el mismo nombre y FIRMA (lista/tipo de parametros) del metodo original
public override void asignarfechanac(string anno, string mes, string dia){

    TimeSpan diftiempo; //almacena un rango de tiempo entre 2 fechas

    try//detecta si la fecha no tiene el formato esperado
    {
        DateTime fnac = Convert.ToDateTime(anno + "/" + mes + "/" + dia);

        //calcula diferencia de tiempo entre fecha de nac. recibida y la fecha actual
        diftiempo = fnac - DateTime.Now;

        //determina si fecha ingresada es al menos 18 años antes de la fecha actual
        if (diftiempo.TotalDays < -365*18)
            fechanac = fnac; //acepta fecha recibida en parametros
        else
            MessageBox.Show("Fecha nacimiento debe ser al menos 18 años antes de hoy", "ERROR-Sistema",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    catch (FormatException e)
    {
        MessageBox.Show("Fecha brindada es incorrecta","ERROR-SISTEMA!!",MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}
```

```
//Sobrecarga de metodo:
public void asignarfechanac(string fecha)
{
    DateTime fechafin;

    //Funcion intenta convertir cadena fecha a tipo DateTime en fechafin.
    //Si es correcto, devuelve true
    if(DateTime.TryParse(fecha,out fechafin)){
        fechanac = fechafin;
    }else
        MessageBox.Show("Fecha de nacimiento es recibida", "ERROR DE SISTEMA!!", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
}

//Confirma si datos para agregar un Doctor son correctos
public bool DatossonCorrectos()
{
    if (nombre != "")
    if (sueldo > 0)
    if (fechanac != Convert.ToDateTime("1900/01/01"))
        return (true); //Los datos recibidos son correctos
    else
        MessageBox.Show("aun no se ha asignado fecha de nacimiento");
    else
        MessageBox.Show("Sueldo incorrecto");
    else
        MessageBox.Show("Nombre aun no asignado");

    return (false);
}
} //fin clase clsDoctor
```

16. Asegurese que no exista ningún error hasta este paso.

17. Luego del cierre de la ultima clase, redacte la definición siguiente de la otra clase (clsVendedor), que heredara también de la clase clsPersona.

```
class clsVendedor : clsPersona
{
    //CAMPOS (propios de clase derivada)
    DateTime fechacontratac;

    //METODOS
    public clsVendedor() //metodo constructor
    {
        fechanac = Convert.ToDateTime("1950/01/01");
        fechacontratac = Convert.ToDateTime("2000/01/01");
    }

    //Este metodo en la clase Hija reemplaza al metodo de la clase Base
    //->Observe que tiene el mismo nombre, pero diferente firma (lista de parametros)
    public new void asignarfechanac(DateTime fechanacimiento)
    {
        fechanac = fechanacimiento;
    }
}
```

```
//PROPIEDADES

public DateTime FechaContrato {
    get {
        return (fechacontratac);
    }
    set
    {
        this.fechacontratac = value;
    }
}

} //fin clase Vendedor
```

18. Corrija todos los errores que puede tener hasta ahora.

Parte 3: Implementacion lógica del formulario

19. Ahora se procederá a modificar el código de la clase del formulario, para utilizar objetos de las clases derivadas (clsDoctor y clsVendedor).
20. De clic en cualquier parte vacia del formulario. Colocar el cursor a nivel del comienzo de la clase Form2. Ahí digite el siguiente código:

```
//Declara a 2 listas/colecciones, basadas en objetos de ambas clases derivadas
List<clsDoctor> doctores;
List<clsVendedor> vendedores;

//prepara objeto para generar cuadro de dialogo de seleccion de archivo
OpenFileDialog cuadroseleccion = new OpenFileDialog();

//METODOS de la clase Form1

//Metodo para generar las secuencias de dias, meses y años en combobox
private void llenarcombobox() {
    int c; //contador usos varios
    //primero dias
    for (c = 1; c <= 31; c++)
        cmbDia.Items.Add(c);
        cmbDia.Text = DateTime.Now.Day.ToString();

    for (c = 1; c <= 12; c++) cmbMes.Items.Add(c);
    cmbMes.Text = DateTime.Now.Month.ToString();

    for (c = 1950; c <= DateTime.Now.Year; c++) cmbAnno.Items.Add(c);

    cmbAnno.Text = DateTime.Now.Year.ToString();
} //fin metodo llenarcombox

void PrepararNuevoDoctor()
{
    //Deja los controles listos para el proximo doctor a ingresar
```

```

txtNom.Clear();
picFotoDoc.ImageLocation = Application.StartupPath + "\\Desconocido.jpg";
cmbDia.SelectedIndex = 0;
cmbMes.SelectedIndex = 0;
cmbAnno.SelectedIndex = 0;
nudSueldo.Value = 1.00M;
mtbCodDoctor.Clear();

txtNom.Focus();
}

void PrepararNuevoVendedor()
{
//deja los controles listos para el proximo vendedor a ingresar
txtNomV.Clear();
picFotoVen.ImageLocation = Application.StartupPath + "\\Desconocido.jpg";

dtpFechaNac.Value = DateTime.Now;
dtpFechaC.Value = DateTime.Now;

txtNomV.Focus();
}

public void MostrarLista_Doctores()
{
//Recorre coleccion doctores, para mostrar datos en datagrid
int cm = 0;
int i;

//se ajusta el titulo de la ultima columna de grilla
dataGridView1.Columns[4].HeaderText = "Codigo";

dataGridView1.Rows.Clear();

//se llena la grilla con filas para los doctores registrados
for(i=0; i<doctores.Count;i++)
    dataGridView1.Rows.Add();

foreach(DataGridViewRow fila in dataGridView1.Rows)
{
//De cada objeto Doctor de la List, extrae valores de sus campos privados
fila.Cells[0].Value = cm+1;
fila.Cells[1].Value = doctores[cm].nombreempleado;
fila.Cells[2].Value = Image.FromFile(doctores[cm].URLfoto);
fila.Cells[3].Value = doctores[cm].fecha_nacimiento;
fila.Cells[4].Value = doctores[cm].codigodoctor;

cm++;
}
}

public void MostrarLista_Vendedores()
{
//recorre coleccion vendedores, para mostrar datos en datagrid
int cm = 0;
int i;

```

```
//Ajusta el titulo de la ultima columna de grilla
dataGridView1.Columns[4].HeaderText = "Fecha Contrato";

dataGridView1.Rows.Clear();

//se llena la grilla con filas para los vendedores registrados
for (i = 0; i < vendedores.Count; i++)
    dataGridView1.Rows.Add();

foreach (DataGridViewRow fila in dataGridView1.Rows)
{
    fila.Cells[0].Value = cm + 1;
    fila.Cells[1].Value = vendedores[cm].nombreempleado;
    fila.Cells[2].Value = Image.FromFile(vendedores[cm].URLfoto);
    fila.Cells[3].Value = vendedores[cm].fecha_nacimiento;
    fila.Cells[4].Value = vendedores[cm].FechaContrato;

    cm++;
}
}
```

21. Digite los siguientes segmentos de código en el control y proc. de evento indicado en el encabezado de c/bloque.

Control: Form1, evento: Load

```
llenarcombobox(); //llena los 3 combos (que definen a una fecha)

//crea las listas de doctores y vendedores
doctores = new List<clsDoctor>();
vendedores = new List<clsVendedor>();

//Define filtros para el cuadro de seleccion, que mostrara solo imagenes JPG
cuadroseleccion.Filter = "Imágenes de tipo JPG |*.jpg";

//Inicia los controles para ingresar un nuevo Doctor o un nuevo Vendedor
PrepararNuevoDoctor();
PrepararNuevoVendedor();
```

Control: btnBuscar, evento: Click

```
//Se invoca un objeto de clase OpenFileDialog, para que usuario seleccione archivo
//de imagen y así, se cargue en el cuadro de imagen
if (cuadroseleccion.ShowDialog() == DialogResult.OK)
{
    picFotoDoc.ImageLocation = cuadroseleccion.FileName;
}
```

Control: btnAgregarDoc, evento: Click

```
clsDoctor Doctor = new clsDoctor(); //objeto de la clase derivada

//toma los datos de los controles del form para definir a objeto Doctor
Doctor.nombreempleado = txtNom.Text;
```

```

Doctor.asignarfechanac(cmbAnno.Text, cmbMes.Text, cmbDia.Text);
Doctor.sueldobase = Convert.ToSingle(nudSueldo.Value);
Doctor.codigodoctor = mtbCodDoctor.Text;

//envia url de foto seleccionada por usuario
Doctor.URLfoto = picFotoDoc.ImageLocation;

//intenta agregar nuevo doctor a coleccion de Doctores
if (Doctor.DatossonCorrectos())
{
    doctores.Add(Doctor);
    MessageBox.Show("Doctor " + Doctor.nombreempleado + " se ha agregado a la
lista", "Excelente!!", MessageBoxButtons.OK, MessageBoxIcon.Information);

//Inicia los controles para ingresar un nuevo Doctor
    PrepararNuevoDoctor();
}
else
    MessageBox.Show("Datos incompletos");

```

Control: btnBuscar2, evento: Click

```

//Selecciona un archivo de imagen para el vendedor
if (cuadroseleccion.ShowDialog() == DialogResult.OK)
{
    picFotoVen.ImageLocation = cuadroseleccion.FileName;
}

```

Control: btnAgregarVen, evento: Click

```

clsVendedor Vendedor = new clsVendedor(); //objeto de la clase derivada
//Define valores brindados por usuario hacia objeto vendedor
Vendedor.nombreempleado = txtNomV.Text;
Vendedor.sueldobase = 2000; //asigna un sueldo fijo de $2000;
Vendedor.asignarfechanac(dtpFechaNac.Value);
Vendedor.FechaContrato = dtpFechaC.Value;

//envia url de foto seleccionada por usuario
Vendedor.URLfoto = picFotoVen.ImageLocation;

//agrega vendedor actual hacia list de vendedores
vendedores.Add(Vendedor);

//deja controles listos para el prox. vendedor a registrar
PrepararNuevoVendedor();

```

Control: rbDoctores, evento: CheckedChanged

```

//invoca metodo para llenar al Datagrid con datos de los doctores registrados
MostrarLista_Doctores();

```

Control: rbVendedores, evento: CheckedChanged

```

//invoca metodo para llenar al Datagrid con datos de los vendedores registrados
MostrarLista_Vendedores();

```

22. Corrija los errores existentes y guarde los cambios en el proyecto.

Parte 4: Pruebas de funcionamiento de la aplicación

23. Para poder continuar, acceda a internet, para descargar imágenes de fotos de personajes, de preferencia en orientación vertical.
Guarde las fotos desde una ubicación que sea fácil de ubicar, como por ej. Dentro de una carpeta creada en Mis Imágenes u otro lugar.
24. Retorne a C# y ejecute la aplicación. Llene los datos solicitados para el 1er doctor.
Presione botón Buscar, para ubicar la carpeta con sus imágenes del paso anterior y asignarle una foto a su gusto.
25. Cuando este listo, presione botón para agregar doctor. Si todo es correcto, vera una confirmación originada desde el interior del objeto Doctor.
26. Seleccione la 3er ficha y presione el 1er radiobutton. Se mostrara los datos del único doctor registrado hasta ahora.
27. Retorne a la ficha 1 y agregue unos 3 doctores mas, asignando una foto diferente a cada uno.
28. Retorne a la ficha 3 y vuelva a ver el listado de doctores.
29. De manera similar, agregue unos 4 vendedores y confirme en la ficha 3 que al seleccionar el 2do radio button, se muestran sus datos.
Presione el 1er radio button y confirme que vuelve a mostrarse la lista de doctores.
30. Agregue otro doctor, pero deje datos incompletos o incompletos en otros controles e intente agregarlo. Vera diferentes mensajes de error originados desde la clase derivada.
31. Por ultimo, explore la carpeta de su proyecto de solución, para ingresar en la siguiente ruta de carpetas:

```
..Guia6Ejercicio1\Guia6Ejercicio1\bin\Debug
```


Observe que la aplicación ha ido creando una copia de cada archivo jpg seleccionado, cambiando de nombre según el nombre del empleado asignado al mismo.
Ahí encontrara también el archivo de la imagen que se muestra cuando aún no ha sido seleccionada una imagen para el empleado.
32. Guarde los cambios realizados al proyecto y muestre el funcionamiento a su instructor(a)

V. ANALISIS DE RESULTADOS

PROBLEMA A RESOLVER

1. Realice una aplicación informática que administre los datos de los diferentes residentes que habitan en un zoológico.
2. Para facilitar la clasificación de los datos, se hará uso de un *esquema de clases*, que se ajustará a la siguiente descripción:
 - A. Crear una clase abstracta denominada **Animales**, la cual tendrá los siguientes datos comunes a todas las especies de animales:
 - ✓ Nombre común
 - ✓ Nombre científico
 - ✓ Foto
 - ✓ Familia (por ejemplo: carnívoro, ungulados, primates, pájaros, gallinácea, insectos, arácnidos, crustáceos, etc.)
 - ✓ Hábitat (por ejemplo: selva, desierto, mar, río, lagos, etc....)
 - ✓ (Fecha de registro/última actualización)
 - B. De la clase anterior, se deberán crear clases derivadas que permitan registrar la siguiente información específica de cada residente del zoológico:
 - Mamífero: se clasifican por su desarrollo embrionario (monotremas, marsupiales y placentarios) y cantidad de mamas.
 - Reptil: su orden (lagartos, cocodrilos, ofidios, queloneos) y por el tipo de locomoción (patas, garras, rastreros).
 - Peces: se clasifican por su sistema esquelético (oseos, cartilaginosos, Actinopterygios) y su desarrollo embrionario (ovíparos, vivíparos)
 - Artrópodos: que debe clasificarlos por tipo terrestre o marino, por su cantidad de patas y cantidad de antenas.
3. Para el registro de los datos, se hará uso del esquema de clases diseñado en numeral anterior, creando una serie de Listas, una por cada tipo de clase derivada.
4. Diseñe una interfaz (pantalla) que facilite a los administradores del zoológico el registro de datos de cada animal que llegue ahí y las actualizaciones posteriores de sus datos.

Importante:

 - Diseñar la aplicación de manera tal que el usuario deba escribir lo menos posible, dándole prioridad al uso del ratón.
 - Permita a los usuarios ver un informe de los animales ya registrados en el sistema, mostrando en un único datagridview a los datos registrados por cada animal.

VI. BIBLIOGRAFÍA

- Bill Evjen, Jason Beres y otros (2002), El libro de Visual Basic.NET (1era Ed), España. Editorial Anaya
- Ceballos, Francisco Javier (2006), ENCICLOPEDIA DE MICROSOFT VISUAL BASIC (1er Ed), Mexico, Editorial ALFAOMEGA, ISBN: 9701512189